

Conceptos y Paradigmas de Lenguajes de Programación - Seg Parcial 1ra Fecha. T2 - 13/06/2025

Realice el parcial con lapicera, de otra forma se desaprobará el/los ejercicio/s.

Se considera presentismo cuando se realiza completamente un ejercicio.

Legajo: 14350/7				Corrigió: JULIAN	
Apellido y Nombres: Pedro Carballer					
1	a	B	b	B	
2	a	B	b	B	
3	a	M	b	B	
4	a	R	b	B	c B d B e B
Resultado Final					Aprobado

Ejercicio 1 (30p)

Realice la pila de ejecución para el siguiente código:

a) por cadena estática b) por cadena dinámica **Nota:** La forma de evaluación de este lenguaje es de izquierda a derecha.

Program Main; Var h, a, b, c: integer; m, n: array[1..3] of integer; Procedure Uno(val-res a: integer; nombre b: integer); var h: integer; begin h:=5; a:= f + 2; b:= b + 3; c:=1; h:= a + 10; end; Function F: integer; Var b: integer; Begin b:=1+h; if(a < 3 and h > 0) then begin n(1):= n(1) + 1; a:= a + 1; b:=2 end; end; end;	if (b < 3) then begin m(b):=m(b)+n(1)-2; m(b)=m(b) * 2; b:=1; end else begin b:=3; end return m(b); end //de la función F begin //del main a:= 1; b:= 1; h:=1; for c:=1 to 3 do begin m(c):= c+1; n(c):= c * 2; end; Uno(m(c), n(c)); for c:=1 to 3 do write (m(c), n(c)); end.
---	--

Ejercicio 2

a) (10pts) Clasifique las siguientes estructuras de datos de acuerdo a lo visto en la práctica. Justifique en cada caso:

i) C <pre>typedef union { int i; float f; char texto[100]; } Valor;</pre>	ii) Java <pre>String[] piezas = {"tuerca", "tornillo", "clavo"}; List<Herramienta> herramientas = new ArrayList<>(); herramientas.add(new Herramienta(1, "tuerca")); herramientas.add(new Herramienta(2, "tornillo")); herramientas.add(new Herramienta(3, "clavo")); herramientas.add(new Herramienta(4, "martillo")); herramientas.add(new Herramienta(5, "destornillador"));</pre>
--	---

b) (10 pts) Responda si las siguientes afirmaciones son V o F. Justifique en cada caso

- i) Los chequeos dinámicos no son obstáculo para que un lenguaje sea fuertemente tipado.
- iii) Los paradigmas funcional y lógico son equivalentes, sólo difieren en la sintaxis.

Ejercicio 3

a) (15 pts) Dado el siguiente código en Java, establezca cuáles de las opciones indicadas más abajo son válidas como camino de ejecución. Justifique su selección con una breve descripción del flujo de ejecución, caso contrario no se considerará válida la respuesta)


```

1 public class Tema2Excepciones {
2     public static void main(String[] args) {
3         try{
4             for (int i = 0; i < 5; i++) {
5                 if(i==1){
6                     System.out.println(Integer.toString(i));
7                     relanzador(i);
8                 }
9                 else{
10                    if(i==2 || i==3){
11                        switch(i){
12                            case 2:
13                                System.out.println(Integer.toString(i));
14                                relanzador(i);
15                                break;
16                            case 3:
17                                System.out.println(Integer.toString(i));
18                                relanzador(i);
19                                break;
20                        }
21                    }
22                }
23                else{
24                    System.out.println(Integer.toString(i));
25                    relanzador(i);
26                }
27            }
28        }
29        catch(ThirdException | FourthException e){
30            System.out.println(e.getMessage());
31        }
32    }
33
34    static void relanzador(int i) throws ThirdException, FourthException {
35        try {
36            try {
37                switch(i){
38                    case 1:
39                        throw new FirstException(Integer.toString(i));
40                        break;
41                    case 2:
42                        throw new SecondException(Integer.toString(i));
43                        break;
44                    case 3:
45                        throw new ThirdException(Integer.toString(i));
46                        break;
47                    default:
48                        throw new FourthException(Integer.toString(i));
49                        break;
50                }
51            }
52            catch (SecondException e) {
53                ThirdException e1=new ThirdException(Integer.toString(i));
54                throw e1;
55            }
56            catch (FirstException e) {
57                FourthException e1=new FourthException(Integer.toString(i));
58                throw e1;
59            }
60        }

```

0
relanzador(0).
FourthException(0)

1
relanzador(1)

2
relanzador(2)

2

- i) Se imprime en pantalla "0", "0", "1", "1", "2", "2", "3", "3", "4", "4" y luego termina
 ii) Se imprime en pantalla "0", luego "0" y termina
 iii) Se imprime en pantalla "0", "0", "1", "1", "2", "2", "3", "3" y luego termina
 iv) Ninguna de las anteriores

b) (15 pts) Indique si eliminando el break de la línea 40 el resultado es el mismo. Indique qué se imprime en ese caso

Ejercicio 4

(20pts). Marcar si son verdaderas o falsas las siguientes afirmaciones. Acompañar la respuesta con una justificación, caso contrario, NO se tomarán como válidas

- a. El case de ADA es más seguro que el de Pascal
 b. Los hechos en un lenguaje lógico son equivalentes a las funciones del paradigma funcional
 c. El tipo unión discriminada se define en base al uso de recursión
 d. En PL/1 si se genera una excepción, la misma se propaga dinámicamente a quien invocó al módulo generador de la excepción
 e. La sentencia else de python es equivalente al finally, salvo que se ejecuta cuando no se ejecutó ninguno de los excepts definidos en el bloque try

V F
V F
V F
V F
V F

1) a) CADENA ESTÁTICA

Pedro Carballo
14350/7

Página 1 de 4

Main
↓
Una
↓
F

valor resultado
→ m[3]

#1	Reg Act Main
	PR
	LE = 1
	LD = 1
	h = 1
	a = 2
	b = 2
	c = 3
	m[1] = 2
	m[2] = 8
	m[3] = 4
	n[1] = 3
	n[2] = 4
	n[3] = 9
	Procedure Una
	Function F
	VR

#2	Reg Act Una
	PR
	LE = 1
	LD = 1
	h = 14
	a = 4
	b = n(c)
	VR = 2
	b = b + 3
	n(b) = n(c) + 3
	n(3) = n(3) + 3
	= 6 + 3
	= 9

#3	Reg Act F
	PR
	LE = 1
	LD = 2
	b = 2
	VR
	m(b) = m(b) + n(1) - 2
	m(2) = m(2) + n(1) - 2
	m(2) = 3 + 3 - 2
	= 4
	m(b) = m(b) + 2
	m(2) = m(2) + 2
	= 4 + 2
	= 6

imprime

2, 3

8, 4

4, 9

b) CADENA DINÁMICA

Main
1
Uno
1
F

valor-retornado
x m[3]

*1 Reg Act Main

pn
LE
LD
h = 1
a = 1
b = 1
c = ~~1~~ ~~3~~ ~~1~~ ~~3~~
m[1] = 2
m[2] = 3
m[3] = ~~4~~ 6
n[1] = 2
n[2] = 4
n[3] = ~~6~~ 9
Procedure Uno
Function F
Vn

*2 Reg Act Uno

pn
LE = *1
LD = *1
h = ~~8~~ 16
a = ~~4~~ 6
b = $\uparrow n(c)$
Vn = 4

 $n(c) = n(c) + 3$
 $n(3) = n(3) + 3$
 $= 6 + 3$
 $= 9$

*3 Reg Act F

pn
LE = *1
LD = *2
b = ~~6~~ 3
Vn

imprime

2,2

3,4

6,9

2) a)

Hoy 2 de 4 Pedro Carballo
14350/17

i) En este ejemplo puede verse la estructura de union, ya que, es una estructura de datos excluyentes uno del otro (en este caso 3 tipos de datos)

Además puede apreciarse la estructura de correspondencia finita, ya que, hay un arreglo de 100 posiciones, y cumple con la definición de correspondencia finita: función de un conjunto de valores finito de tipo de dato de dominio (DT) en un conjunto de valores resultado (RT). ✓

ii) En este ejemplo puede verse correspondencia finita en "piezas"; ya que, también sigue la definición de correspondencia finita: función de un conjunto de valores finito de tipo de dato de dominio (DT) en un conjunto de valores resultado (RT). ✓

Además, se puede apreciar un Arraylist, lo que en Java se considera un TAD (por su encapsulamiento y su funcionamiento).

b) i) Verdadero.

Esta afirmación es verdadera, ya que, por ejemplo Python es un lenguaje con chequeos dinámicos y a su vez se considera fuertemente tipado. ✓

ii) Falso.

Esta afirmación es falsa. El paradigma lógico se basa en aserciones lógicas compuestas de reglas y hechos, y el paradigma funcional no. Además, por ejemplo el concepto de variables en el paradigma funcional es el concepto de variables matemáticas (inmutables), y en el paradigma lógico no es así. ✓

4) a) Verdadero.
El caso de ADA es más seguro que el de Pascal ya que la variable utilizada para esa estructura no puede ser modificada en ADA. ADA además es un lenguaje más seguro que Pascal.
R, uso de "OTHERS"

b) Falso.
Los funciones de un paradigma funcional devuelven un valor, los hechos en un lenguaje lógico no.

c) Falso.
El tipo unión discriminada es una estructura de campos excluyentes el uno del otro pero no se depende en base al uso de recursión (cuya definición es un tipo de dato recursivo de tipo T contiene a dato de tipo T).

d) Falso.
En PL/I si se genera una excepción se busca un manejador para esa excepción en la pila (de orden LIFO) que previamente debe haberse enlazado utilizando ON CONDITION ... BEGIN ... END;

e) Falso.
La sentencia else de Python se ejecuta únicamente cuando no se genera ninguna excepción dentro del bloque try.
En cambio la sentencia finally se ejecuta siempre.

3) La opción correcta es la i.

Pedro Carballo
14350/7

M, ES LA ii

Página 3 de 4

El flujo de ejecución de este código comienza en el main, en la primera iteración del for ($i=0$), entra al else de la línea 23, imprime "0" y llama a relanzador con el valor 0 pasado como parámetro.

En relanzador, se entra al default en la línea 47 y se lanza una excepción de tipo `FourthException` enviando "0" como parámetro. Al no encontrar el manejador correspondiente en relanzador, se busca dinámicamente y se maneja en la línea 29 del main. Allí se imprime "0".

Termina el bloque donde se generó la excepción, y se vuelve a la línea 26 del main. **→ NO SE VUELVE, TERMINA**

Se pasa a la segunda iteración del for con $i=1$, se entra al if de la línea 6, se imprime "1" y se llama a relanzador con "1" como parámetro.

En relanzador se entra al case de la línea 38, se lanza una excepción de tipo `FirstException` con "1" como parámetro. Se encuentra el manejador de manera estática en el bloque try de arriba (en ~~relanzador~~ relanzador) y se crea una excepción de tipo `FourthException` con "1" como parámetro y se lanza. Se busca dinámicamente el manejador y se ejecuta la línea 29 del main. Se maneja la excepción y se imprime "1". Luego termina el bloque donde se generó la excepción y se vuelve

al main a la próxima iteración del for.

Con $i=2$, se entra al if de la línea 11, luego al case de la línea 13, se imprime "2".

y se llama a relanzador con 2 como parámetro.

En relanzador se entra al case de la línea 41, lanzando una excepción de tipo SecondException con "2" como parámetro. Se busca el manejador y se encuentra en el catch línea 52. Se lanza una excepción de tipo ThirdException con "2" como parámetro y se maneja en el catch del main línea 29. Se imprime "2".

Se termina el bloque try donde se generó la excepción y se vuelve al main.

Iteración del for con $i=3$.

entra al case de la línea 17. Se imprime "3".
se llama ~ relanzador
pasando 3 como parámetro, se genera relanzador, línea 44. Se lanza ThirdException con "3" como parámetro. Se maneja esta excepción en main línea 29. Se imprime "3".

Similar a lo anterior, se termina el bloque donde se generó la excepción, se vuelve al main y sigue el for.

Iteración $i=4$. entra al else línea 23, se imprime "4" y se llama a relanzador

Se ajusta el default linea 77, lanza excepción
de tipo ~~throw~~ Exception con "4" como parámetro
Se maneja esa excepción en linea 29 del main
Se imprime "4".

Termina el bloque try donde se lanzó la excepción,
vuelve al main, i pasa a valor 5 y termina.

b) Eliminando el break de la linea 40 el resultado
no cambia ya que una vez que se lanza una
excepción, se termina el bloque donde esto ocurrió.

Rodolfo Cardillo

14/3/2017

Página 4 de 4